

Déploiement srsRAN 4G avec ZMQ Virtual Radios



1. Objectif du projet

L'objectif de cette SAE est de déployer une infrastructure mobile basée sur le projet open-source **srsRAN**. Dans notre cas, le travail demandé concerne la mise en place d'un réseau **4G LTE** à l'aide des entités **srsUE**, **srsENB** et **srsEPC**. Ces entités doivent être déployées sur des machines virtuelles afin de tester le fonctionnement du système. Le document du projet précise que le réseau LTE de base doit être réalisé avec **srsUE** sur une première machine et **srsENB** avec **srsEPC** sur une deuxième machine virtuelle.

2. Architecture mise en place

Nous avons utilisé deux machines virtuelles Ubuntu 22.04.

Machine	Rôle	Services
VM1	Terminal utilisateur LTE	srsUE
VM2	Réseau opérateur	srsENB + srsEPC

L'architecture utilisée est donc la suivante :

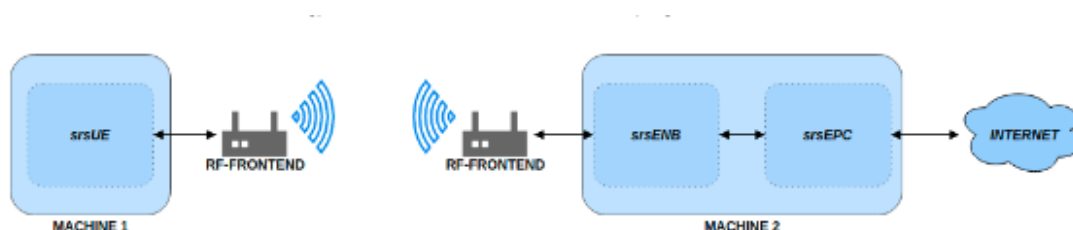


FIGURE 1 : Réseau srsRAN LTE de base

Dans l'architecture :

- **srsUE** simule l'équipement utilisateur, comme un téléphone LTE.
- **srsENB** simule l'antenne LTE, appelée eNodeB.
- **srsEPC** simule le cœur de réseau LTE.
- ZeroMQ/ZMQ permet de simuler le lien radio sans utiliser de matériel radio physique.

3. Configuration réseau des machines virtuelles

Chaque machine virtuelle possède deux cartes réseau.

Interface	Rôle
enp0s3	Réseau interne / RT-Box / communication entre les deux VMs
enp0s8	NAT / accès Internet pour les installations

Les adresses IP obtenues étaient :

machine	interface	Adresse IP
VM1	enp0s3	192.31.25.14/24
VM1	enp0s3	10.0.3.15/24
VM2	enp0s8	192.31.25.12/24
VM2	enp0s8	10.0.3.15/24

J'ai ensuite testé la communication entre les deux machines avec la commande **ping**.

Depuis VM1 :

ping 192.31.25.12

```
administrateur@rt-mv:~/Téléchargements/srsRAN_4G-master/build$ ping 192.31.25.12
PING 192.31.25.12 (192.31.25.12) 56(84) bytes of data.
64 bytes from 192.31.25.12: icmp_seq=1 ttl=64 time=0.232 ms
64 bytes from 192.31.25.12: icmp_seq=2 ttl=64 time=0.226 ms
64 bytes from 192.31.25.12: icmp_seq=3 ttl=64 time=0.246 ms
^C
```

Depuis VM2 :

ping 192.31.25.14

```
administrateur@rt-mv:~$ ping 192.31.25.14
PING 192.31.25.14 (192.31.25.14) 56(84) bytes of data.
64 bytes from 192.31.25.14: icmp_seq=1 ttl=64 time=0.231 ms
64 bytes from 192.31.25.14: icmp_seq=2 ttl=64 time=0.429 ms
^C
--- 192.31.25.14 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1029ms
rtt min/avg/max/mdev = 0.231/0.330/0.429/0.099 ms
administrateur@rt-mv:~$
```

Les deux tests ont fonctionné, ce qui a permis de valider la communication entre les machines virtuelles.

4. Correction du problème de route par défaut

Nous avons rencontré un problème réseau : la route par défaut repassait régulièrement par l'interface `enp0s3`, c'est-à-dire par la RT-Box, au lieu de passer par l'interface NAT `enp0s8`.

La mauvaise route était :

```
default via 192.31.25.1 dev enp0s3
```

Pour corriger temporairement le problème, nous avons utilisé:

```
sudo ip route del default via 192.31.25.1 dev enp0s3
```

Ensuite, nous avons configuré NetworkManager afin que l'interface `enp0s3` ne soit pas utilisée comme route par défaut.

Commandes:

```
sudo nmcli connection modify "Connexion filaire 1" ipv4.never-default yes
sudo nmcli connection modify "Connexion filaire 2" ipv4.never-default no
sudo nmcli connection modify "Connexion filaire 2" ipv4.route-metric 100
sudo systemctl restart NetworkManager
```

Après cette configuration, l'accès Internet via `apt update` fonctionnait correctement sur les deux VMs.

5. Installation des dépendances

Sur les deux machines virtuelles, nous avons installé les dépendances nécessaires à la compilation et au fonctionnement de srsRAN 4G.

Commande utilisée :

```
sudo apt install -y git cmake make gcc g++ libfftw3-dev libmbedtls-dev libsctp-dev
libyaml-cpp-dev libgtest-dev libboost-program-options-dev libconfig++-dev iproute2 net-tools
libzmq3-dev unzip curl
```

Ces dépendances nous aide notamment :

- de compiler le projet avec `cmake`, `make`, `gcc` et `g++` ;
- d'utiliser les bibliothèques nécessaires à srsRAN ;
- d'utiliser ZeroMQ grâce au paquet `libzmq3-dev` ;
- de décompresser l'archive ZIP du projet avec `unzip`.

6. Téléchargement de srsRAN 4G

Le téléchargement avec `git clone` pas fonctionné à cause de restrictions réseau liées à l'environnement de l'école.

L'erreur obtenue était liée à l'accès à GitHub ou à la résolution DNS.

Nous avons donc téléchargé le projet manuellement depuis GitHub avec Firefox :

[GitHub](#) → [srsran/srsRAN_4G](#) → [Code](#) → [Download ZIP](#)

Le fichier ZIP a ensuite été téléchargé dans le dossier :

`~/Téléchargements`

7. Décompression du projet

Sur chaque VM, nous sommes allés dans le dossier de téléchargement :

`cd ~/Téléchargements`

Puis nous avons vérifié la présence du fichier :

`ls`

Ensuite, nous avons décompressé l'archive :

`unzip srsRAN_4G-master.zip`

Puis nous sommes entrés dans le dossier du projet :

`cd srsRAN_4G-master`

8. Compilation de srsRAN 4G

J'ai créé un dossier `build` pour compiler le projet.

Commandes utilisées :

```
mkdir build
cd build
cmake ../
make -j$(nproc)
```

La commande `cmake ../` prépare la compilation.

La commande suivante lance la compilation en utilisant tous les cœurs disponibles :

```
make -j$(nproc)
```

9. Installation de srsRAN 4G

Après la compilation, nous avons installé srsRAN avec :

```
sudo make install  
sudo ldconfig
```

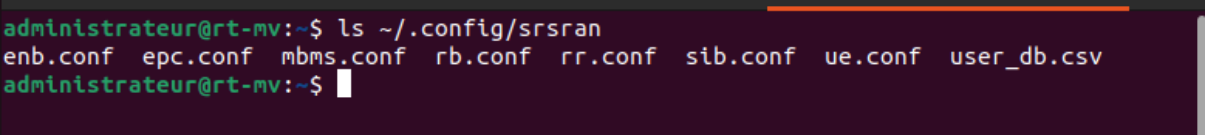
Ensuite, nous avons installé les fichiers de configuration utilisateur avec:

```
srsran_install_configs.sh user
```

Nous avons vérifié la présence des fichiers de configuration avec:

```
ls ~/.config/srsran
```

Les fichiers obtenus étaient :



```
administrateur@rt-mv:~$ ls ~/.config/srsran  
enb.conf epc.conf mbms.conf rb.conf rr.conf sib.conf ue.conf user_db.csv  
administrateur@rt-mv:~$
```

f

10. Vérification de l'installation

Nous avons vérifié que les commandes srsRAN étaient bien disponibles.

Commandes utilisées :

```
srsue --version  
srsenb --version  
srsepc --version
```

Pour VM2

```

administrateur@rt-mv:~$ srsue --version
srseNB --version
srsePC --version
Active RF plugins: libsrslan_rf_zmq.so
Inactive RF plugins:
Version 25.10.0
Active RF plugins: libsrslan_rf_zmq.so
Inactive RF plugins:
--- Software Radio Systems LTE eNodeB ---

Version 25.10.0

Built in Release mode using 25.10.0.

--- Software Radio Systems EPC ---

Couldn't open , trying /home/administrateur/.config/srslan/epc.conf
Reading configuration file /home/administrateur/.config/srslan/epc.conf...
Couldn't open user_db.csv, trying /home/administrateur/.config/srslan/user_db.csv
srslan error - Unable to create log file "/tmp/epc.log": Permission denied
HSS Initialized.
bind(): Address already in use
Error binding SCTP socket
administrateur@rt-mv:~$

```

Pour VM1

```

administrateur@rt-mv:~/Téléchargements/srslan_4G-master/build$ srsue --version
srseNB --version
srsePC --version
Active RF plugins: libsrslan_rf_zmq.so
Inactive RF plugins:
Version 25.10.0
Active RF plugins: libsrslan_rf_zmq.so
Inactive RF plugins:
--- Software Radio Systems LTE eNodeB ---

Version 25.10.0

Built in Release mode using 25.10.0.

--- Software Radio Systems EPC ---

Couldn't open , trying /home/administrateur/.config/srslan/epc.conf
Reading configuration file /home/administrateur/.config/srslan/epc.conf...
Couldn't open user_db.csv, trying /home/administrateur/.config/srslan/user_db.csv
HSS Initialized.
MME S11 Initialized
MME GTP-C Initialized
MME Initialized. MCC: 0xf001, MNC: 0xff01
Could not initialize the SGI interface.
Could not initialize the SPGW's GTP-U.
Error initializing SP-GW
administrateur@rt-mv:~/Téléchargements/srslan_4G-master/build$

```

La version installée était :

Version 25.10.0

Nous avons aussi vérifié que le plugin ZeroMQ était actif :

Active RF plugins: libsrslan_rf_zmq.so

Cela confirme que srsRAN peut utiliser ZMQ comme interface radio virtuelle.

11. Configuration de l'EPC sur VM2

L'EPC est lancé sur VM2.

Avant de lancer le service, j'ai supprimé les anciens fichiers logs qui pouvaient causer des erreurs de permissions :

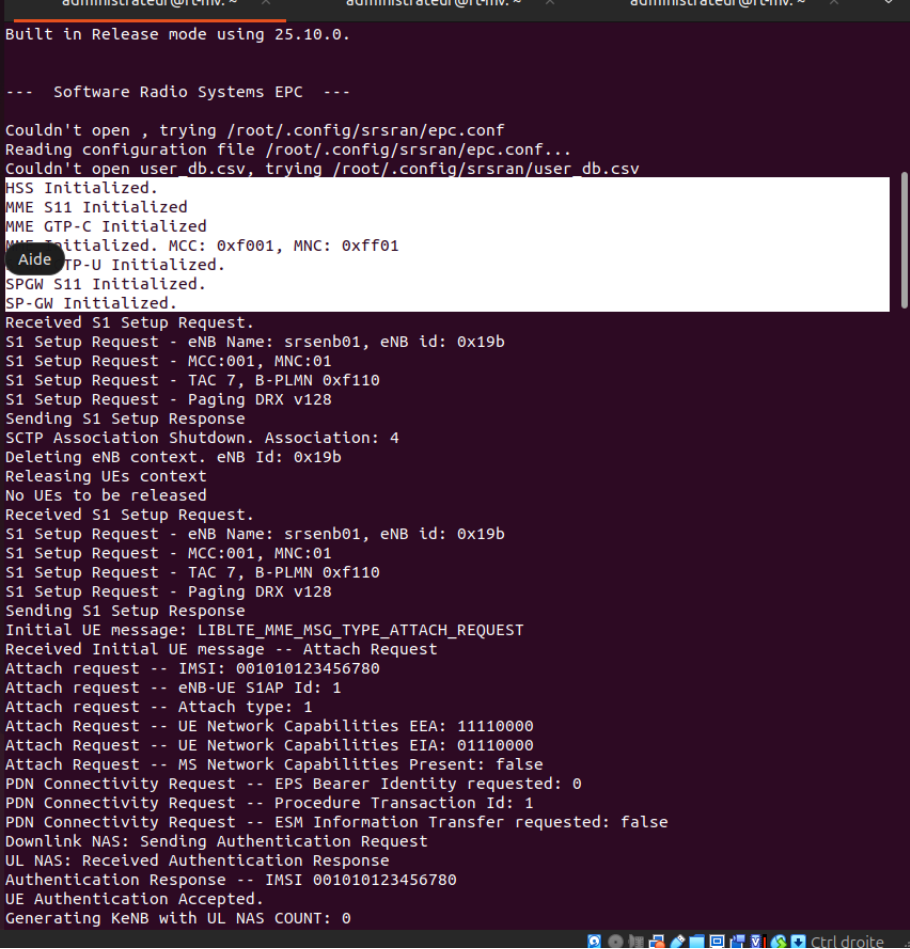
```
sudo rm -f /tmp/epc.log
```

```
sudo rm -f /tmp/enb.log
```

```
sudo rm -f /tmp/ue.log
```

Puis nous avons lancé l'EPC :

```
sudo srsepc
```



```
Built in Release mode using 25.10.0.

--- Software Radio Systems EPC ---

Couldn't open , trying /root/.config/srsran/epc.conf
Reading configuration file /root/.config/srsran/epc.conf...
Couldn't open user db.csv, trying /root/.config/srsran/user_db.csv
HSS Initialized.
MME S11 Initialized
MME GTP-C Initialized
MME Initialized. MCC: 0xf001, MNC: 0xff01
MME GTP-U Initialized.
SPGW S11 Initialized.
SP-GW Initialized.
Received S1 Setup Request.
S1 Setup Request - eNB Name: srsenb01, eNB id: 0x19b
S1 Setup Request - MCC:001, MNC:01
S1 Setup Request - TAC 7, B-PLMN 0xf110
S1 Setup Request - Paging DRX v128
Sending S1 Setup Response
SCTP Association Shutdown. Association: 4
Deleting eNB context. eNB Id: 0x19b
Releasing UEs context
No UEs to be released
Received S1 Setup Request.
S1 Setup Request - eNB Name: srsenb01, eNB id: 0x19b
S1 Setup Request - MCC:001, MNC:01
S1 Setup Request - TAC 7, B-PLMN 0xf110
S1 Setup Request - Paging DRX v128
Sending S1 Setup Response
Initial UE message: LIBLTE_MME_MSG_TYPE_ATTACH_REQUEST
Received Initial UE message -- Attach Request
Attach request -- IMSI: 001010123456780
Attach request -- eNB-UE S1AP Id: 1
Attach request -- Attach type: 1
Attach Request -- UE Network Capabilities EEA: 11110000
Attach Request -- UE Network Capabilities EIA: 01110000
Attach Request -- MS Network Capabilities Present: false
PDN Connectivity Request -- EPS Bearer Identity requested: 0
PDN Connectivity Request -- Procedure Transaction Id: 1
PDN Connectivity Request -- ESM Information Transfer requested: false
Downlink NAS: Sending Authentication Request
UL NAS: Received Authentication Response
Authentication Response -- IMSI 001010123456780
UE Authentication Accepted.
Generating KeNB with UL NAS COUNT: 0
```

L'EPC s'est correctement initialisé avec les messages suivants :

HSS Initialized.

MME S1 Initialized.

MME GTP-C Initialized.

SPGW GTP-U Initialized.
SPGW S11 Initialized.
SP-GW Initialized.

Ces messages montrent que le cœur de réseau LTE est bien démarré.

12. Configuration de l'eNodeB sur VM2

L'eNodeB doit utiliser ZMQ comme interface radio.

Nous avons modifié le fichier :

```
sudo nano /root/.config/srsran/enb.conf
```

Dans la section [\[rf\]](#), nous avons configuré :

```
[rf]
device_name = zmq
device_args =
fail_on_disconnect=true,tx_port=tcp://*:2000,rx_port=tcp://192.31.25.14:2001,id=enb,base_s
rate=23.04e6
```

```
GNU nano 6.2 /root/.config/srsran/enb.conf
# eNB configuration files
#
# sib_config: SIB1, SIB2 and SIB3 configuration file
# note: When enabling MBMS, use the sib.conf.mbsfn configuration file which includes SIB13
# rr_config: Radio Resources configuration file
# rb_config: SRB/DRB configuration file
#####
[enb_files]
sib_config = sib.conf
rr_config = rr.conf
rb_config = rb.conf

#####
# RF configuration
#
# dl_earfcn: EARFCN code for DL (only valid if a single cell is configured in rr.conf)
# tx_gain: Transmit gain (dB).
# rx_gain: Optional receive gain (dB). If disabled, AGC is enabled
#
# Optional parameters:
# dl_freq: Override DL frequency corresponding to dl_earfcn
# ul_freq: Override UL frequency corresponding to dl_earfcn (must be set if dl_freq is set)
# device_name: Device driver family
#             Supported options: "auto" (uses first driver found), "UHD", "bladeRF"
# device_args: Arguments for the device driver. Options are "auto" or any string.
#             Default for UHD: "recv_frame_size=9232,send_frame_size=9232"
#             Default for bladeRF: ""
# time_adv_nsamples: Transmission time advance (in number of samples) to compensate for RF delay
#                   from antenna to timestamp insertion.
#                   Default "auto". B210 USRP: 100 samples, bladeRF: 27
#####
[rf]
device_name = zmq
device_args = fail_on_disconnect=true,tx_port=tcp://*:2000,rx_port=tcp://192.31.25.14:2001,

#dl_earfcn = 3350
tx_gain = 80
rx_gain = 40

#device_name = auto

# For best performance in 2x2 MIMO and >= 15 MHz use the following device_args settings:
# USRP B210: num_recv_frames=64,num_send_frames=64

^G Aide      ^O Écrire    ^W Chercher  ^K Couper    ^T Exécuter  ^C Emplacement
^X Quitter   ^R Lire fich.^_ Remplacer  ^U Coller    ^J Justifier  ^_ Aller ligne
Ctrl droite
```

Explication :

- `device_name = zmq` indique que l'on utilise ZeroMQ.
- `tx_port=tcp://*:2000` indique le port d'émission de l'eNodeB.
- `rx_port=tcp://192.31.25.14:2001` indique que l'eNodeB reçoit les données venant de VM1, où se trouve srsUE.
- 192.31.25.14 correspond à l'adresse IP de VM1.
- `id=enb` identifie l'équipement comme eNodeB.
- `base_srate=23.04e6` définit le taux d'échantillonnage utilisé.

Ensuite, nous avons lancé l'eNodeB :

```
sudo srsenb
```

Le démarrage a été validé par le message :

=== eNodeB started ===

```
==== eNodeB started ===
Type <t> to view trace
Current sample rate is 11.52 MHz with a base rate of 23.04 MHz (x2 decimation)
Current sample rate is 11.52 MHz with a base rate of 23.04 MHz (x2 decimation)
Setting frequency: DL=2680.0 Mhz, UL=2560.0 Mhz for cc_idx=0 nof_prb=50
RACH: tti=181, cc=0, pci=1, preamble=23, offset=0, temp_crnti=0x46
User 0x46 connected
Disconnecting rnti=0x46.
RACH: tti=2751, cc=0, pci=1, preamble=42, offset=0, temp_crnti=0x47
User 0x47 connected
Disconnecting rnti=0x47.
RACH: tti=9511, cc=0, pci=1, preamble=40, offset=0, temp_crnti=0x48
User 0x48 connected
Disconnecting rnti=0x48.
```

Côté EPC, nous avons aussi observé :

Received S1 Setup Request
Sending S1 Setup Response

Cela confirme que **srsENB** communique correctement avec **srsEPC**.

13. Configuration de l'UE sur VM1

Sur VM1, nous avons configuré **srsUE** pour utiliser ZMQ.

Fichier modifié :

`sudo nano /root/.config/srsran/ue.conf`

Dans la section **[rf]**, nous avons configuré :

```
[rf]
device_name = zmq
device_args =
tx_port=tcp://*:2001,rx_port=tcp://192.31.25.12:2000,id=ue,base_srate=23.04e6
```

```
# continuous_tx:    Transmit samples continuously to the radio or on bursts (auto/yes/no).
#                  Default is auto (yes for UHD, no for rest)
#####
[rf]
device_name = zmq
device_args = tx_port=tcp://*:2001,rx_port=tcp://192.31.25.12:2000,id=ue,base_srate=23.04e6
freq_offset = 0
tx_gain = 80
#rx_gain = 40
#srate = 11.52e6
```

Explication :

- `device_name = zmq` permet d'utiliser ZeroMQ.
- `tx_port=tcp://*:2001` indique le port d'émission de l'UE.
- `rx_port=tcp://192.31.25.12:2000` indique que l'UE reçoit les données venant de VM2.
- `192.31.25.12` correspond à l'adresse IP de VM2.
- `id=ue` identifie l'équipement comme UE.
- `base_srate=23.04e6` doit correspondre à celui utilisé côté eNodeB.

14. Lancement des services dans le bon ordre

Pour que le réseau fonctionne, les services doivent être lancés dans cet ordre.

Sur VM2 — Terminal 1

`sudo srsepc`

Attendre :

SP-GW Initialized.

```
administrateur@rt-mv:~$ sudo rm -f /tmp/epc.log /tmp/enb.log /tmp/ue.log
sudo srsepc

Built in Release mode using 25.10.0.

--- Software Radio Systems EPC ---

Couldn't open , trying /root/.config/srsran/epc.conf
Reading configuration file /root/.config/srsran/epc.conf...
Couldn't open user_db.csv, trying /root/.config/srsran/user_db.csv
HSS Initialized.
MME S11 Initialized
MME GTP-C Initialized
MME Initialized. MCC: 0xf001, MNC: 0xff01
SPGW GTP-U Initialized.
SPGW S11 Initialized.
SP-GW Initialized.
```

Sur VM2 — Terminal 2

`sudo srsenb`

Attendre :

=== eNodeB started ===

```
[sudo] Mot de passe de administrateur :
Active RF plugins: librsran_rf_zmq.so
Inactive RF plugins:
--- Software Radio Systems LTE eNodeB ---

Couldn't open , trying /root/.config/srsran/enb.conf
Reading configuration file /root/.config/srsran/enb.conf...
Couldn't open sib.conf, trying /root/.config/srsran/sib.conf
Couldn't open rr.conf, trying /root/.config/srsran/rr.conf
Couldn't open rb.conf, trying /root/.config/srsran/rb.conf

Built in Release mode using 25.10.0.

Opening 1 channels in RF device=zmq with args=fail_on_disconnect=true,tx_port=tcp://*:2000,rx_port=tcp://192.31.25.14:2001,id=enb,base_srate=23.04e6
Supported RF device list: zmq file
CHx base_srate=23.04e6
CHx id=enb
Current sample rate is 1.92 MHz with a base rate of 23.04 MHz (x12 decimation)
CH0 rx_port=tcp://192.31.25.14:2001
CH0 tx_port=tcp://*:2000
CH0 fail_on_disconnect=true

==== eNodeB started ====
```

Sur VM1 — Terminal 1

sudo srsue

Attendre :

Network attach successful. IP: 172.16.0.2

```
administrateur@rt-mv:~/Téléchargements/srsRAN_4G-master/build$ sudo srsue
Active RF plugins: librsran_rf_zmq.so
Inactive RF plugins:
Couldn't open , trying /root/.config/srsran/ue.conf
Reading configuration file /root/.config/srsran/ue.conf...

Built in Release mode using 25.10.0.

Opening 1 channels in RF device=zmq with args=tx_port=tcp://*:2001,rx_port=tcp://192.31.25.12:2000,id=ue,base_srate=23.04e6
Supported RF device list: zmq file
CHx base_srate=23.04e6
CHx id=ue
Current sample rate is 1.92 MHz with a base rate of 23.04 MHz (x12 decimation)
CH0 rx_port=tcp://192.31.25.12:2000
CH0 tx_port=tcp://*:2001
Waiting PHY to initialize ... done!
Attaching UE...
Current sample rate is 1.92 MHz with a base rate of 23.04 MHz (x12 decimation)
Current sample rate is 1.92 MHz with a base rate of 23.04 MHz (x12 decimation)
.
Found Cell: Mode=FDD, PCI=1, PRB=50, Ports=1, CP=Normal, CFO=-0.2 KHz
Current sample rate is 11.52 MHz with a base rate of 23.04 MHz (x2 decimation)
Current sample rate is 11.52 MHz with a base rate of 23.04 MHz (x2 decimation)
Found PLMN: Id=00101, TAC=7
Random Access Transmission: seq=23, tti=181, ra-rnti=0x2
RRC Connected
Random Access Complete. c-rnti=0x46, ta=0
Network attach successful. IP: 172.16.0.2
```

15. Résultat obtenu côté UE

1. Côté VM1, **srsUE** a détecté la cellule LTE :
2. Found Cell: Mode=FDD, PCI=1, PRB=50, Ports=1, CP=Normal
3. Puis le PLMN :

4. Found PLMN: Id=00101, TAC=7
5. Ensuite, l'UE s'est attaché au réseau :
6. RRC Connected
7. Random Access Complete
8. Network attach successful. IP: 172.16.0.2
9. Ce résultat confirme que l'UE s'est connecté au réseau LTE simulé avec ZMQ.

16. Vérification de l'interface LTE sur VM1

Après l'attachement de l'UE, nous avons ouvert un deuxième terminal sur VM1 et lancé :

`ip a`

Nous avons observé l'interface :

`tun_srsue`

avec l'adresse IP :

`172.16.0.2/24`

Cela confirme que l'UE a bien reçu une adresse IP via le réseau LTE.

```
administrateur@rt-mv:~/Téléchargements/srsRAN_4G-master/build$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:91:b4:ff brd ff:ff:ff:ff:ff:ff
    inet 192.31.25.14/24 brd 192.31.25.255 scope global dynamic noprefixroute enp0s3
        valid_lft 307sec preferred_lft 307sec
    inet6 fe80::fa3b:1e7c:f7bc:ac41/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:07:1c:bb brd ff:ff:ff:ff:ff:ff
    inet 10.0.3.15/24 brd 10.0.3.255 scope global dynamic noprefixroute enp0s8
        valid_lft 80707sec preferred_lft 80707sec
    inet6 fd17:625c:f037:3:75d2:8c7b:db29:9c87/64 scope global temporary dynamic
        valid_lft 86268sec preferred_lft 14268sec
    inet6 fd17:625c:f037:3:6c9e:426b:512b:a203/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 86268sec preferred_lft 14268sec
    inet6 fe80::e3:95cf:1089:350f/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
6: tun_srsue: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN group default qlen 500
    link/none
    inet 172.16.0.2/24 scope global tun_srsue
        valid_lft forever preferred_lft forever
administrateur@rt-mv:~/Téléchargements/srsRAN_4G-master/build$
```

17. Test de connectivité avec l'EPC

Depuis VM1, nous avons testé la communication avec l'interface SGi de l'EPC :

`ping 172.16.0.1`

Résultat obtenu :

```
administrateur@rt-mv:~/Téléchargements/srsRAN_4G-master/build$ ping 172.16.0.1
PING 172.16.0.1 (172.16.0.1) 56(84) bytes of data.
64 bytes from 172.16.0.1: icmp_seq=1 ttl=64 time=235 ms
64 bytes from 172.16.0.1: icmp_seq=2 ttl=64 time=24.3 ms
64 bytes from 172.16.0.1: icmp_seq=3 ttl=64 time=33.8 ms
64 bytes from 172.16.0.1: icmp_seq=4 ttl=64 time=36.4 ms
^C
--- 172.16.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3002ms
rtt min/avg/max/mdev = 24.337/82.297/234.631/88.064 ms
administrateur@rt-mv:~/Téléchargements/srsRAN_4G-master/build$
```

Cela valide la communication entre :

srsUE $\Rightarrow$ srsENB $\Rightarrow$ srsEPC

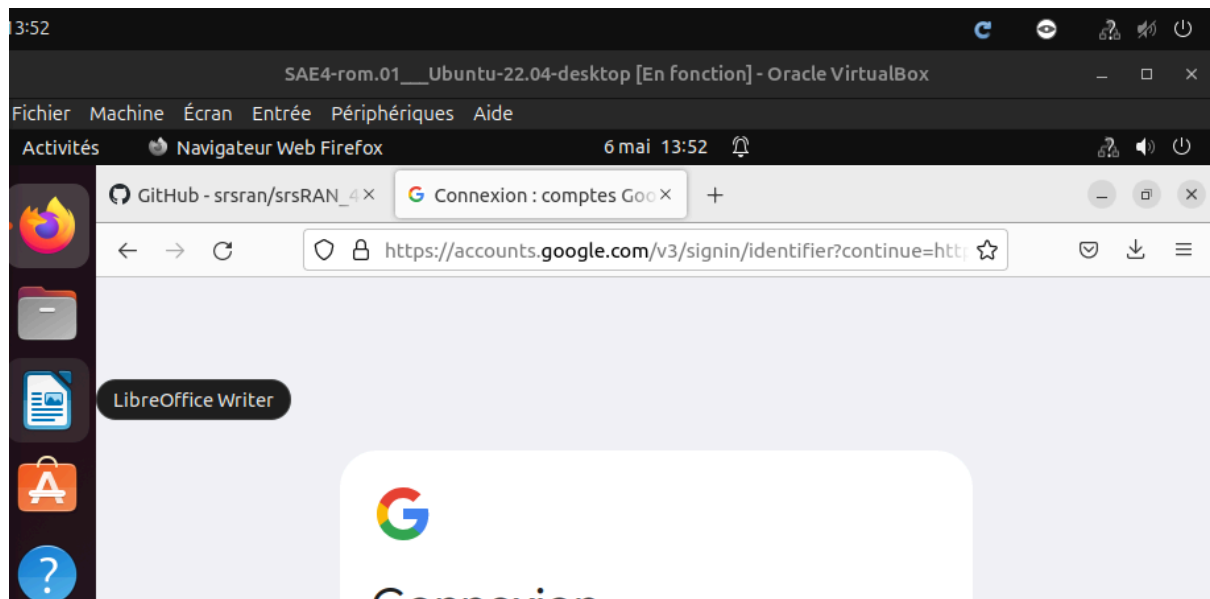
18. Test Internet

Nous avons ensuite testé l'accès vers Internet depuis l'UE avec la commande suivante :

`ping -I tun_srsue 8.8.8.8`

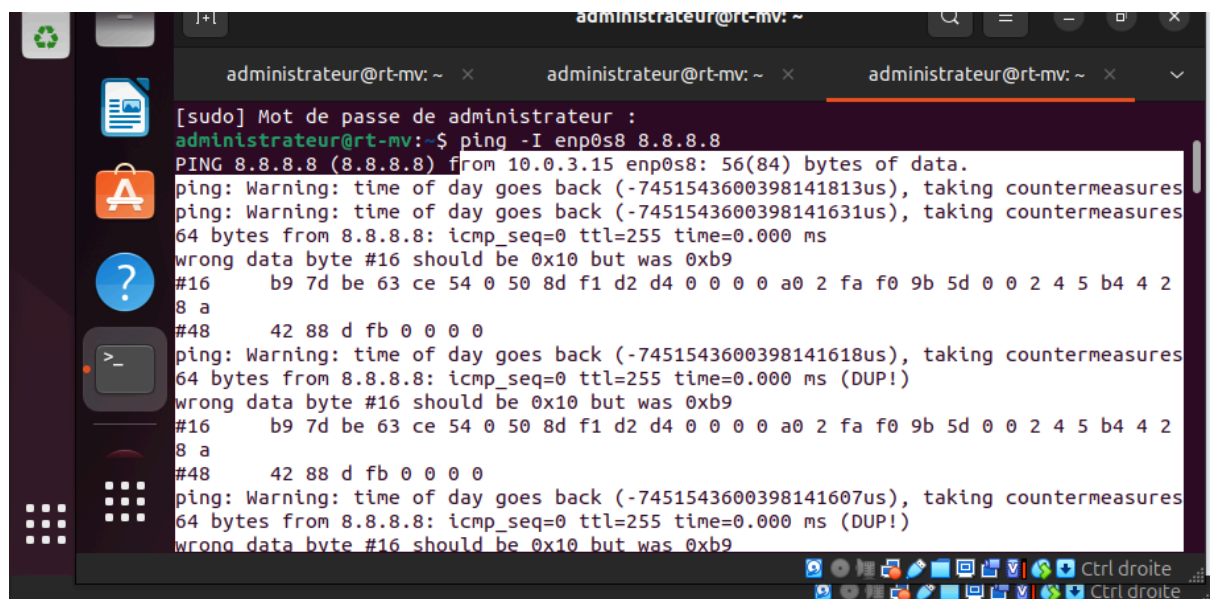
```
Corbeille ttd_lft 86268sec preferred_lft 14268sec
fd17:625c:f037:3:6c9e:426b:512b:a203/64 scope global dynamic mngtmpaddr noprefixro
ute
    valid_lft 86268sec preferred_lft 14268sec
    inet6 fe80::e3:95cf:1089:350f/64 scope link noprefixroute
    valid_lft forever preferred_lft forever
6: tun_srsue: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN
N group default qlen 500
    link/none
    inet 172.16.0.2/24 scope global tun_srsue
    valid_lft forever preferred_lft forever
administrateur@rt-mv:~/Téléchargements/srsRAN_4G-master/build$ ping 172.16.0.1
PING 172.16.0.1 (172.16.0.1) 56(84) bytes of data.
64 bytes from 172.16.0.1: icmp_seq=1 ttl=64 time=235 ms
64 bytes from 172.16.0.1: icmp_seq=2 ttl=64 time=24.3 ms
64 bytes from 172.16.0.1: icmp_seq=3 ttl=64 time=33.8 ms
64 bytes from 172.16.0.1: icmp_seq=4 ttl=64 time=36.4 ms
^C
--- 172.16.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3002ms
rtt min/avg/max/mdev = 24.337/82.297/234.631/88.064 ms
administrateur@rt-mv:~/Téléchargements/srsRAN_4G-master/build$ ping -I tun_srsue 8.8.8.8
PING 8.8.8.8 (8.8.8.8) from 172.16.0.2 tun_srsue: 56(84) bytes of data.
```

Ce test ICMP pas donné de réponse. mais, cela signifie pas que l'accès Internet est totalement impossible. En effet, dans l'environnement réseau de IUT, l'accès au Web est possible via un navigateur comme Firefox, mais les requêtes ICMP utilisées par la commande `ping` semblent être bloquer

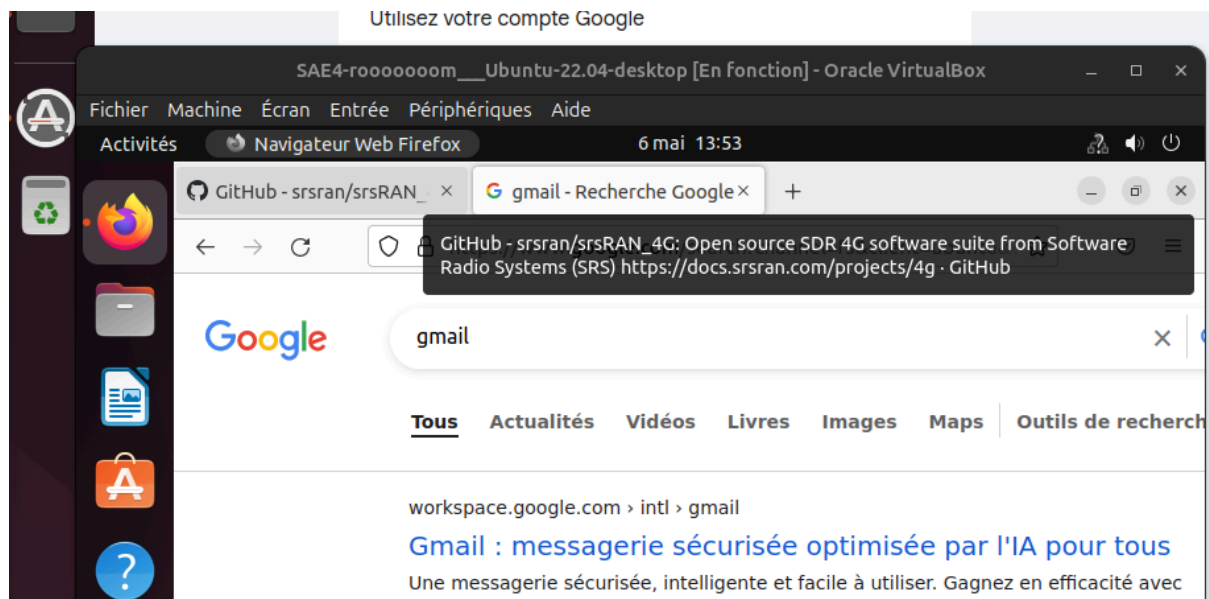


Nous avons également constaté que le même comportement apparaît directement depuis VM2 avec la commande :

`ping -I enp0s8 8.8.8.8`



Ainsi, le problème ne vient pas de la configuration LTE/ZMQ, mais plutôt des règles de filtrage réseau de l'établissement, qui autorisent la navigation Web tout en bloquant certains tests réseau comme le ping.



Cette observation permet de conclure que le réseau LTE interne est bien fonctionnel, puisque l'UE obtient une adresse IP LTE et communique correctement avec l'EPC via l'adresse **172.16.0.1**.

19. Activation du routage/NAT sur VM2

Nous avons quand même configuré VM2 pour permettre le routage entre le réseau LTE et l'interface NAT.

Commandes utilisées sur VM2 :

```
sudo sysctl -w net.ipv4.ip_forward=1
```

Puis :

```
sudo iptables -t nat -A POSTROUTING -s 172.16.0.0/24 -o enp0s8 -j MASQUERADE
```

Puis :

```
sudo iptables -A FORWARD -i srs_spgw_sgi -o enp0s8 -j ACCEPT
sudo iptables -A FORWARD -i enp0s8 -o srs_spgw_sgi -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
```

L'interface EPC observée sur VM2 était :

```
srs_spgw_sgi : 172.16.0.1/24
```

20. Conclusion de l'étape ZMQ

La mise en place de srsRAN 4G avec ZMQ Virtual Radios a été réalisée avec succès. Les trois entités principales, srsUE, srsENB et srsEPC, fonctionnent correctement sur les machines virtuelles. L'UE détecte la cellule LTE, s'attache au réseau et reçoit l'adresse IP 172.16.0.2 sur l'interface tun_srsue. La communication avec l'EPC est validée par un ping vers 172.16.0.1 avec 0% de perte.

21. Carrier Aggregation avec ZeroMQ RF emulation

Après avoir réussi la première partie avec le réseau LTE simple en ZMQ, j'ai passés à la partie **Carrier Aggregation using ZeroMQ RF emulation**.

Le but ici est d'utiliser deux porteuses LTE au lieu d'une seule. Cela permet de simuler l'agrégation de porteuses avec srsRAN, toujours sans utiliser de vraie interface radio.

L'architecture reste la même :

VM1 : srsUE

VM2 : srsENB + srsEPC

La communication radio est encore simulée avec ZeroMQ.

22. Sauvegarde des anciennes configurations

Avant de modifier les fichiers, nous avons fait une sauvegarde des fichiers qui fonctionnaient déjà avec la configuration ZMQ simple.

Sur VM2 :

```
sudo cp /root/.config/srsran/enb.conf /root/.config/srsran/enb.conf.zmq_basic_backup
sudo cp /root/.config/srsran/rr.conf /root/.config/srsran/rr.conf.zmq_basic_backup
```

Sur VM1 :

```
sudo cp /root/.config/srsran/ue.conf /root/.config/srsran/ue.conf.zmq_basic_backup
```

Comme ça, si la configuration Carrier Aggregation ne marchait pas, on pouvait revenir à l'ancienne version.

23. Modification du fichier **enb.conf** sur VM2

Sur VM2, nous avons modifié le fichier de configuration de l'eNodeB :

```
sudo nano /root/.config/srsran/enb.conf
```

Dans la partie [rf], nous avons mis une configuration avec deux ports ZMQ :

```
# device_args:      Arguments for the device driver. Options are "auto" or any str>
#                  Default for UHD: "recv_frame_size=9232,send_frame_size=9232"
#                  Default for bladeRF: ""
# time_adv_nsamples: Transmission time advance (in number of samples) to compensat>
#                  from antenna to timestamp insertion.
#                  Default "auto". B210 USRP: 100 samples, bladeRF: 27
#####
[rf]
device_name = zmq
device_args = fail_on_disconnect=true,id=enb,tx_port0=tcp://*:2000,tx_port1=tcp://*:>

#dl earfcn = 3350
```

Ici, 192.31.25.14 correspond à l'adresse IP de VM1, où se trouve srsUE.

Cette configuration permet à l'eNodeB d'utiliser deux canaux ZMQ, un pour chaque porteuse.

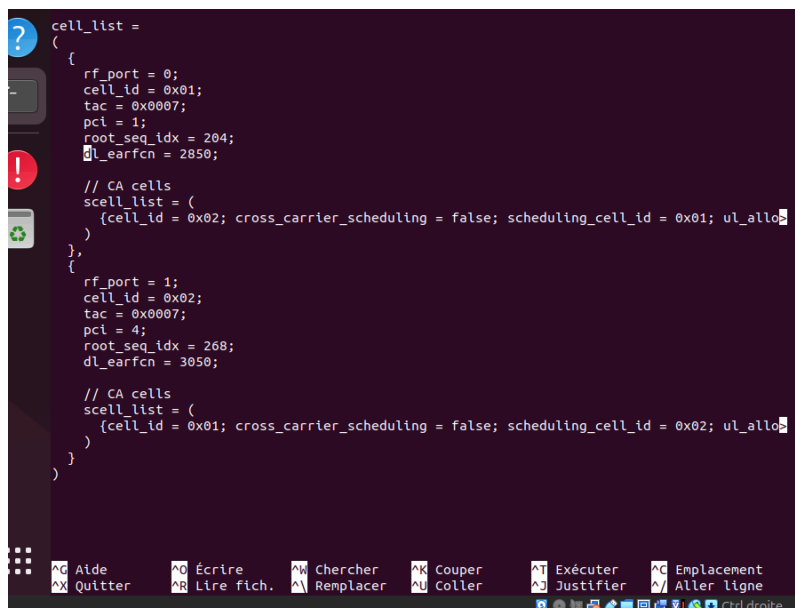
24. Modification du fichier rr.conf sur VM2

Ensuite, nous avons modifié le fichier :

`sudo nano /root/.config/srsran/rr.conf`

Dans ce fichier, nous avons configuré deux cellules LTE. Une cellule sert de cellule principale et l'autre sert de cellule secondaire.

Configuration utilisée :



```
? cell_list =
(
{
  rf_port = 0;
  cell_id = 0x01;
  tac = 0x0007;
  pci = 1;
  root_seq_idx = 204;
  dl_earfcn = 2850;

  // CA cells
  scell_list = (
  {cell_id = 0x02; cross_carrier_scheduling = false; scheduling_cell_id = 0x01; ul_allo
  },
  ),
{
  rf_port = 1;
  cell_id = 0x02;
  tac = 0x0007;
  pci = 4;
  root_seq_idx = 268;
  dl_earfcn = 3050;

  // CA cells
  scell_list = (
  {cell_id = 0x01; cross_carrier_scheduling = false; scheduling_cell_id = 0x02; ul_allo
  }
  )
}
)
```

La première cellule utilise rf_port = 0 et la deuxième utilise rf_port = 1.

Après le lancement de `srsenb`, on voit bien que deux porteuses sont configurées :

```
Setting frequency: DL=2630.0 MHz, UL=2510.0 MHz for cc_idx=0
Setting frequency: DL=2650.0 MHz, UL=2530.0 MHz for cc_idx=1
```

25. Modification du fichier `ue.conf` sur VM1

Sur VM1, nous avons aussi modifié le fichier de configuration de l'UE :

```
sudo nano /root/.config/srsran/ue.conf
```

Dans la partie `[rf]`, nous avons mis :

```
[rf]
device_name = zmq
device_args =
tx_port0=tcp://*:2001,tx_port1=tcp://*:2003,rx_port0=tcp://192.31.25.12:2000,rx_port1=tcp://
192.31.25.12:2002,id=ue,tx_freq0=2510e6,tx_freq1=2530e6,rx_freq0=2630e6,rx_freq1=265
0e6,base_srate=23.04e6
freq_offset = 0
tx_gain = 80
```

```
# Default: auto. 0:10 USRP, 100 samples, 0:10 USRP, 27.
# continuous_tx: Transmit samples continuously to the radio or on bursts (auto/yes/no).
# Default is auto (yes for UHD, no for rest)
#####
[rf]
device_name = zmq
device_args = tx_port0=tcp://*:2001,tx_port1=tcp://*:2003,rx_port0=tcp://192.31.25.12:2000,rx_p>
freq_offset = 0
tx_gain = 80
#rx_gain = 40
#srate = 11.52e6
#ref_antennas = 1
```

Ici, `192.31.25.12` correspond à l'adresse IP de VM2, où se trouve `srsENB`.

Ensuite, dans la partie `[rat.eutra]`, nous avons mis :

```
[rat.eutra]
dl_earfcn = 2850,3050
nof_carriers = 2
```

```
# Optional parameters.
# dl_eurfcn: Override DL frequency corresponding to dl_eurfcn
# ul_eurfcn: Override UL frequency corresponding to dl_eurfcn
# nof_carriers: Number of carriers
#####
[rat.eutra]
dl_eurfcn = 2850,3050
nof_carriers = 2
#nof_carriers = 1
#####
# NR RAT configuration
#
```

Au début, nous avons mis `nof_carriers = 2` dans la mauvaise partie du fichier. Cela donnait cette erreur :

unrecognised option 'rf.nof_carriers'

Après correction, nous avons mis `nof_carriers = 2` dans `[rat.eutra]`, et l'erreur a disparu.

26. Relancement des services

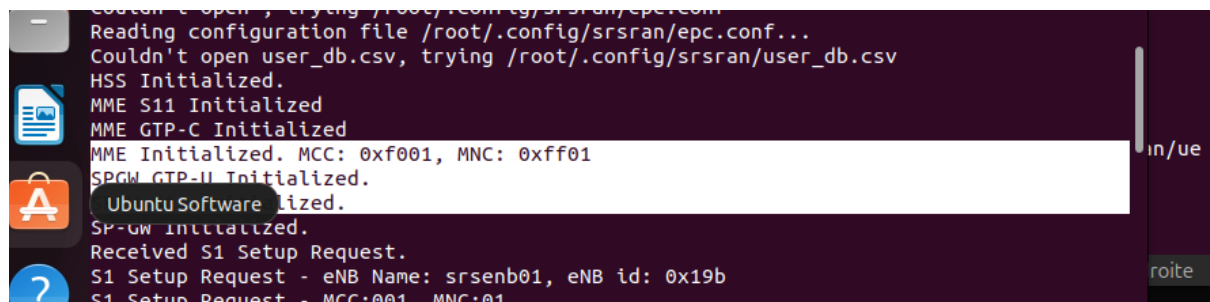
Après les modifications, nous avons relancé les services dans le bon ordre.

Sur VM2, nous avons lancé l'EPC :

`sudo srsepc`

Puis nous avons attendu :

SP-GW Initialized.



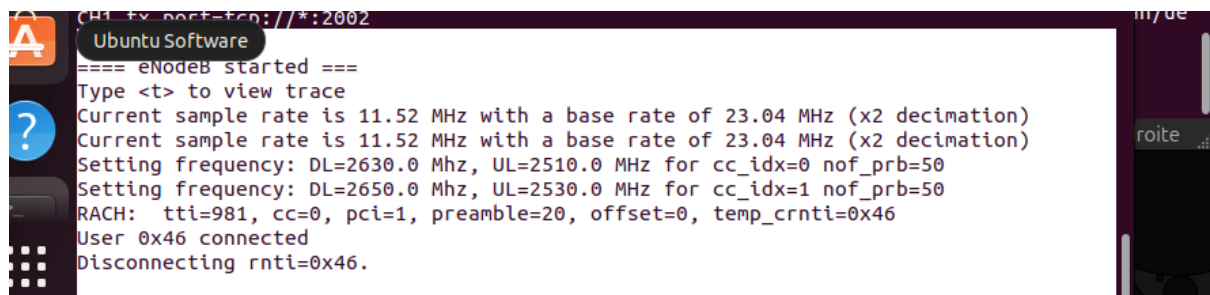
```
Couldn't open , trying /root/.config/srsran/epc.conf...
Reading configuration file /root/.config/srsran/epc.conf...
Couldn't open user_db.csv, trying /root/.config/srsran/user_db.csv
HSS Initialized.
MME S11 Initialized
MME GTP-C Initialized
MME Initialized. MCC: 0xf001, MNC: 0xff01
SPGW GTP-U Initialized.
Ubuntu Software Center Initialized.
SP-GW initialized.
Received S1 Setup Request.
S1 Setup Request - eNB Name: srsenb01, eNB id: 0x19b
S1 Setup Request - MCC: 001 MNC: 01
```

Ensuite, sur VM2 dans un deuxième terminal, nous avons lancé l'eNodeB :

`sudo srsenb`

Nous avons attendu :

=== eNodeB started ===



```
CH1: tx_port=100: //*:2002
Ubuntu Software Center Initialized.
==== eNodeB started ===
Type <t> to view trace
Current sample rate is 11.52 MHz with a base rate of 23.04 MHz (x2 decimation)
Setting frequency: DL=2630.0 Mhz, UL=2510.0 Mhz for cc_idx=0 nof_prb=50
Setting frequency: DL=2650.0 Mhz, UL=2530.0 Mhz for cc_idx=1 nof_prb=50
RACH: tti=981, cc=0, pci=1, preamble=20, offset=0, temp_crnti=0x46
User 0x46 connected
Disconnecting rnti=0x46.
```

Enfin, sur VM1, nous avons lancé l'UE :

`sudo srsue`

```
administrateur@rt-mv: ~/Téléchargements/srsRAN_4G-master/build$ sudo srsue
[sudo] Mot de passe de administrateur :
Active RF plugins: librsran_rf_zmq.so
Inactive RF plugins:
Couldn't open , trying /root/.config/srsran/ue.conf
Reading configuration file /root/.config/srsran/ue.conf...

Built in Release mode using 25.10.0.

Opening 2 channels in RF device=zmq with args=tx_port0=tcp://*:2001,tx_port1=tcp://*:2003,rx_port0=tcp://192.31.25.12:2000,rx_port1=tcp://192.31.25.12:2002,id=ue,tx_freq0=2510e6,tx_freq1=2530e6,rx_freq0=2630e6,rx_freq1=2650e6,base_srate=23.04e6
Supported RF device list: zmq file
CHx base_srate=23.04e6
CHx id=ue
Current sample rate is 1.92 MHz with a base rate of 23.04 MHz (x12 decimation)
CH0 rx_port=tcp://192.31.25.12:2000
CH0 rx_freq=2630e6
CH0 tx_port=tcp://*:2001
CH0 tx_freq=2510e6
CH1 rx_port=tcp://192.31.25.12:2002
CH1 rx_freq=2650e6
CH1 tx_port=tcp://*:2003
Afficher les applications ... done!
Attaching UE...
```

L'ordre est important, car l'EPC doit être lancé avant l'eNodeB, et l'eNodeB doit être lancé avant l'UE.

27. Résultat côté eNodeB

Côté [srsenb](#), nous avons vu que les deux porteuses étaient bien prises en compte :

```
Setting frequency: DL=2630.0 MHz, UL=2510.0 MHz for cc_idx=0 nof_prb=50
Setting frequency: DL=2650.0 MHz, UL=2530.0 MHz for cc_idx=1 nof_prb=50
```

Nous avons aussi vu que l'UE était connecté :

```
User 0x46 connected
```

Cela montre que l'eNodeB fonctionne avec les deux porteuses.

28. Résultat côté UE

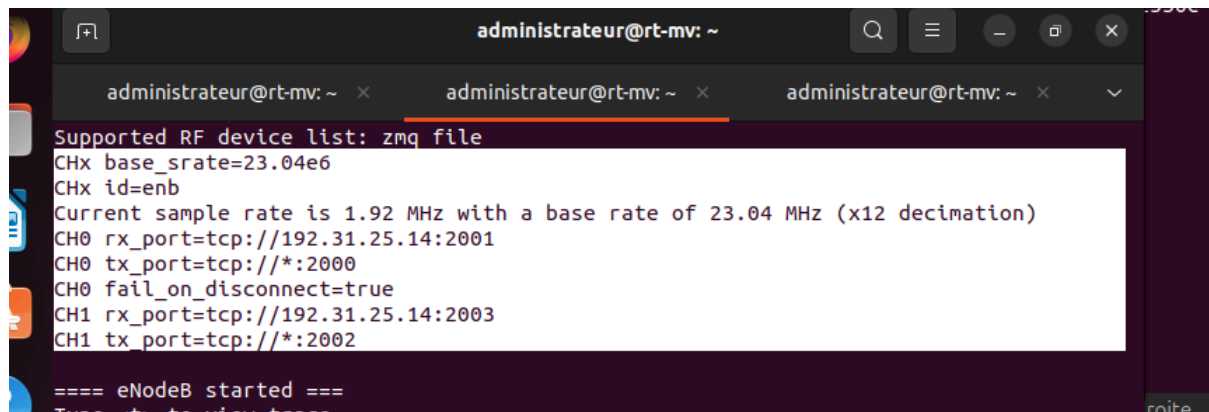
Côté [srsUE](#), nous avons vu que deux canaux RF étaient ouverts :

```
Opening 2 channels in RF device=zmq
```

Ensuite, les deux canaux sont affichés :

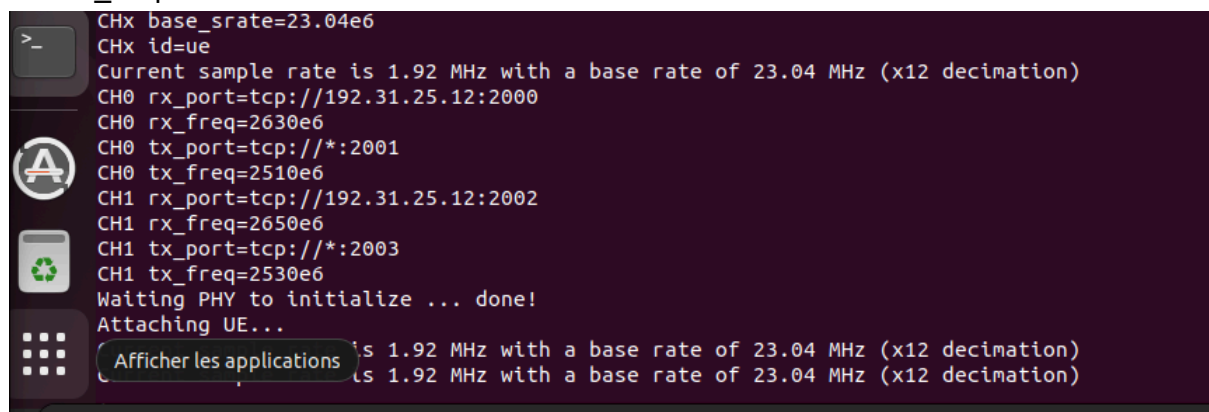
```
CH0 rx_port=tcp://192.31.25.12:2000
```

CH0 rx_freq=2630e6
CH0 tx_port=tcp://*:2001
CH0 tx_freq=2510e6



```
administrateur@rt-mv: ~  
administrateur@rt-mv: ~ x administrateur@rt-mv: ~ x administrateur@rt-mv: ~ x  
Supported RF device list: zmq file  
CHx base_srate=23.04e6  
CHx id=enb  
Current sample rate is 1.92 MHz with a base rate of 23.04 MHz (x12 decimation)  
CH0 rx_port=tcp://192.31.25.14:2001  
CH0 tx_port=tcp://*:2000  
CH0 fail_on_disconnect=true  
CH1 rx_port=tcp://192.31.25.14:2003  
CH1 tx_port=tcp://*:2002  
==== eNodeB started ===  
Type <tab> to view trace
```

CH1 rx_port=tcp://192.31.25.12:2002
CH1 rx_freq=2650e6
CH1 tx_port=tcp://*:2003
CH1 tx_freq=2530e6



```
>_  
CHx base_srate=23.04e6  
CHx id=ue  
Current sample rate is 1.92 MHz with a base rate of 23.04 MHz (x12 decimation)  
CH0 rx_port=tcp://192.31.25.12:2000  
CH0 rx_freq=2630e6  
CH0 tx_port=tcp://*:2001  
CH0 tx_freq=2510e6  
CH1 rx_port=tcp://192.31.25.12:2002  
CH1 rx_freq=2650e6  
CH1 tx_port=tcp://*:2003  
CH1 tx_freq=2530e6  
Waiting PHY to initialize ... done!  
Attaching UE...  
s 1.92 MHz with a base rate of 23.04 MHz (x12 decimation)  
s 1.92 MHz with a base rate of 23.04 MHz (x12 decimation)
```

Après cela, l'UE s'est bien attaché au réseau :

Network attach successful. IP: 172.16.0.2

Nous avons aussi vu cette ligne :

SCELL Activation / Deactivation CMD: 2

Cette ligne est importante, car elle montre que la cellule secondaire a été activée.

29. Vérification de l'interface réseau

Sur VM1, nous avons vérifié les interfaces réseau avec :

`ip a`

L'interface `tun_srsue` était bien présente avec l'adresse :

172.16.0.2

Cela montre que l'UE a bien reçu une adresse IP sur le réseau LTE.

```
link/ether 08:00:27:07:1c:bb brd ff:ff:ff:ff:ff:ff
inet 10.0.3.15/24 brd 10.0.3.255 scope global dynamic noprefixroute enp0s8
    valid_lft 77117sec preferred_lft 77117sec
inet6 fd17:625c:f037:3:75d2:8c7b:db29:9c87/64 scope global temporary dynamic
    valid_lft 86226sec preferred_lft 14226sec
inet6 fd17:625c:f037:3:6c9e:426b:512b:a203/64 scope global dynamic mngtnpaddr noprefixroute
    valid_lft 86226sec preferred_lft 14226sec
inet6 fe80::e3:95cf:1089:350f/64 scope link noprefixroute
    valid_lft forever preferred_lft forever
7: tun_srsue: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN group default qlen 500
    link/none
    inet 172.16.0.2/24 scope global tun_srsue
        valid_lft forever preferred_lft forever
administrateur@rt-mv:~/Téléchargements/srsRAN_4G-master/build$ ping 172.16
PING 172.16 (172.0.0.16) 56(84) bytes of data.
```

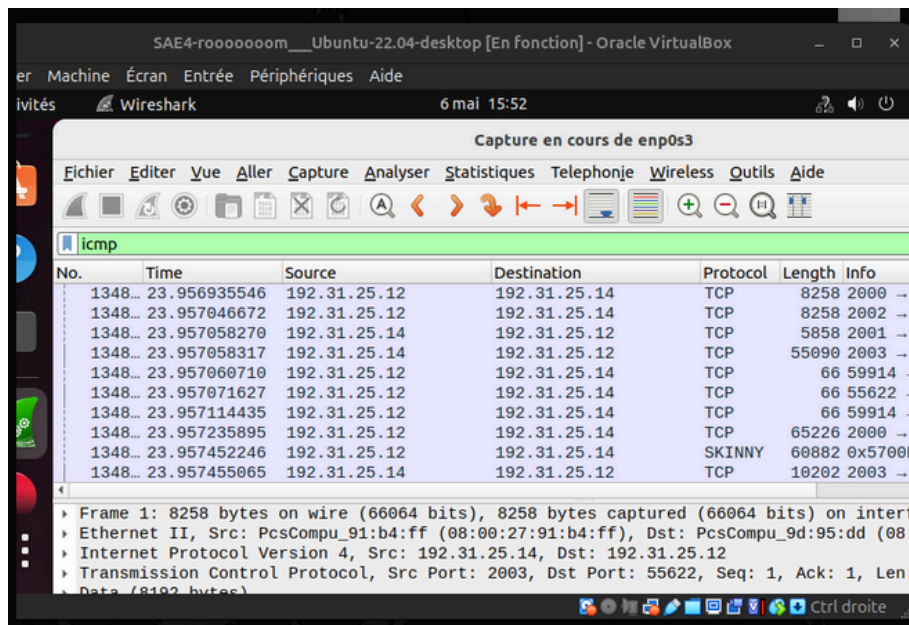
30. Test de connectivité

Pour vérifier la communication avec l'EPC, nous avons fait :

ping 172.16.0.1

```
0
1^C
--- 172.16 ping statistics ---
5 packets transmitted, 0 received, 100% packet loss, time 4095ms

administrateur@rt-mv:~/Téléchargements/srsRAN_4G-master/build$ ping 172.16.0.1
PING 172.16.0.1 (172.16.0.1) 56(84) bytes of data:
64 bytes from 172.16.0.1: icmp_seq=1 ttl=64 time=88.3 ms
64 bytes from 172.16.0.1: icmp_seq=2 ttl=64 time=53.3 ms
64 bytes from 172.16.0.1: icmp_seq=3 ttl=64 time=80.3 ms
^C
--- 172.16.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 53.310/73.983/88.299/14.975 ms
administrateur@rt-mv:~/Téléchargements/srsRAN_4G-master/build$ ping 172.16.0.1
```

Cette capture Wireshark montre les échanges TCP entre VM1 (192.31.25.14) et VM2 (192.31.25.12). Les ports 2000, 2001, 2002 et 2003 correspondent aux ports ZeroMQ utilisés pour simuler le lien radio entre srsUE et srsENB. Cette capture permet donc de vérifier que la communication ZMQ entre les deux machines fonctionne.

31. Problèmes rencontrés

Pendant la réalisation du projet, nous avons rencontré plusieurs problèmes.

Le premier problème était lié à la route par défaut des machines virtuelles. La route passait régulièrement par l'interface `enp0s3`, qui correspond au réseau RT-Box, au lieu de passer par l'interface NAT `enp0s8`. Cela empêchait parfois l'accès Internet pour les commandes comme `apt update`.

Pour corriger ce problème, nous avons supprimé la mauvaise route avec :

```
sudo ip route del default via 192.31.25.1 dev enp0s3
```

Ensuite, nous avons modifié la configuration NetworkManager pour éviter que `enp0s3` soit utilisée comme route par défaut.

```
administrateur@rt-mv:~$ ip route
default via 10.0.3.2 dev enp0s8 proto dhcp metric 20100
10.0.3.0/24 dev enp0s8 proto kernel scope link src 10.0.3.15 metric 100
169.254.0.0/16 dev enp0s3 scope link metric 1000
192.31.25.0/24 dev enp0s3 proto kernel scope link src 192.31.25.14 metric 100
administrateur@rt-mv:~$
```

Un autre problème était le téléchargement du projet srsRAN depuis GitHub. La commande `git clone` fonctionnait pas à cause des restrictions réseau de proxy IUT j'ai donc téléchargé le projet sous forme d'archive ZIP depuis navigateur directement.

J'ai aussi rencontré un problème de place disque pendant l'utilisation de Wireshark. La capture prenait beaucoup d'espace, ce qui a rempli le disque de la machine virtuelle. Pour corriger cela, nous avons supprimé les fichiers temporaires et les anciennes captures inutiles.

Enfin, pendant la configuration de la Carrier Aggregation, nous avons placé au début `nof_carriers = 2` dans la mauvaise section du fichier `ue.conf`. Cela a donné l'erreur :

```
unrecognised option 'rf.nof_carriers'
```

Après correction, nous avons mis ce paramètre dans la section `[rat.eutra]`, ce qui a permis à `srsUE` de démarrer correctement avec deux porteuses.

32. Bilan des résultats obtenus

À la fin de cette partie, nous avons réussi à faire fonctionner un réseau LTE simulé avec srsRAN 4G.

Dans un premier temps, nous avons validé le fonctionnement de base avec ZeroMQ. L'UE a réussi à détecter la cellule LTE, à s'attacher au réseau et à recevoir l'adresse IP `172.16.0.2` sur l'interface `tun_srsue`.

Ensuite, nous avons configuré la Carrier Aggregation avec deux porteuses. Côté eNodeB, les deux porteuses sont visibles avec :

```
cc_idx=0  
cc_idx=1
```

Côté UE, deux canaux RF sont ouverts avec ZeroMQ. Le message suivant confirme aussi que la cellule secondaire a été activée :

SCELL Activation / Deactivation CMD: 2

Le test de connectivité avec l'EPC a aussi fonctionné :

avec `0%` de perte. Cela confirme que la communication entre `srsUE`, `srsENB` et `srsEPC` fonctionne correctement.

```
valid_ttl forever preferred_ttl forever
administrateur@rt-mv:~$ ping 192.168.2.1
PING 192.168.2.1 (192.168.2.1) 56(84) bytes of data.
64 bytes from 192.168.2.1: icmp_seq=1 ttl=64 time=0.569 ms
64 bytes from 192.168.2.1: icmp_seq=2 ttl=64 time=0.730 ms
64 bytes from 192.168.2.1: icmp_seq=3 ttl=64 time=0.388 ms
^C
--- 192.168.2.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2041ms
rtt min/avg/max/mdev = 0.388/0.562/0.730/0.139 ms
administrateur@rt-mv:~$

administrateur@rt-mv:~$ lsusb
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 001 Device 003: ID 0456:b673 Analog Devices, Inc. PlutoSDR (ADALM-PLUTO)
Bus 001 Device 002: ID 80ee:0021 VirtualBox USB Tablet
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
administrateur@rt-mv:~$
```

33.Test avec Adalm-Pluto

Nous avons commencé la partie avec le module **Adalm-Pluto** pour remplacer l'interface radio virtuelle ZMQ par une vraie interface SDR.

Le Pluto a été branché sur le PC puis attaché à **VM2**, la machine qui contient [srsENB](#) et [srsEPC](#).

Nous avons vérifié qu'il était bien détecté avec :

[lsusb](#)

```
administrateur@rt-mv:~$ lsusb
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 001 Device 003: ID 0456:b673 Analog Devices, Inc. PlutoSDR (ADALM-PLUTO)
Bus 001 Device 002: ID 80ee:0021 VirtualBox USB Tablet
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
administrateur@rt-mv:~$
```

Le module apparaît comme :

Analog Devices, Inc. PlutoSDR (ADALM-PLUTO)

Ensuite, avec :

[ip a](#)

nous avons vu une nouvelle interface réseau :

[enx00e0221e5f2e](#) : 192.168.2.10/24

```
4: enx00e0221e5f2e: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state U
    link/ether 00:e0:22:1e:5f:2e brd ff:ff:ff:ff:ff:ff
    inet 192.168.2.10/24 brd 192.168.2.255 scope global dynamic noprefixroute enx00e0
    221e5f2e
```

Le ping vers le Pluto fonctionne :

ping 192.168.2.1

```
valto_ttl forever preferred_ttl forever
administrateur@rt-mv:~$ ping 192.168.2.1
PING 192.168.2.1 (192.168.2.1) 56(84) bytes of data.
64 bytes from 192.168.2.1: icmp_seq=1 ttl=64 time=0.569 ms
64 bytes from 192.168.2.1: icmp_seq=2 ttl=64 time=0.730 ms
64 bytes from 192.168.2.1: icmp_seq=3 ttl=64 time=0.388 ms
^C
--- 192.168.2.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2041ms
rtt min/avg/max/mdev = 0.388/0.562/0.730/0.139 ms
administrateur@rt-mv:~$
```

Enfin, la commande :

`sudo iio_info -u ip:192.168.2.1`

confirme que le matériel est bien reconnu :

hw_model: Analog Devices PlutoSDR Rev.C

uri: ip:192.168.2.1

```
18 device-specific attributes found:
attr 0: burst_count value: 0
attr 1: core_id value: 0
attr 2: enable value: 0
attr 3: frame_length_ms value: 0.000000
attr 4: frame_length_raw value: 0
attr 5: internal_sync_period_ms value: 0.000000
attr 6: internal_sync_period_raw value: 0
attr 7: magic value: 0x5444444e
attr 8: scratch value: 0x00000000
attr 9: startup_delay_ms value: 0.000000
attr 10: startup_delay_raw value: 0
attr 11: state value: 0
attr 12: sync_external value: 0
attr 13: sync_internal value: 0
attr 14: sync_reset value: 0
attr 15: sync_soft ERROR: Permission denied (13)
attr 16: version value: 2.00.b
attr 17: waiting_for_supplier value: 0

No trigger on this device
administrateur@rt-mv:~$
```

Cependant, SoapySDR ne détecte pas encore le Pluto :

`SoapySDRUtil --find`

ne montre que des périphériques audio. De plus, srsRAN affiche seulement le plugin ZMQ :

Active RF plugins: libsrnsan_rf_zmq.so

```
administrateur@rt-mv: ~  
attr 16: version value: 2.00.b  
attr 17: waiting_for_supplier value: 0  
No trigger on this device  
administrateur@rt-mv:~$ SoapySDRUtil --find  
#####  
##      Soapy SDR -- the SDR abstraction library      ##  
#####  
[INFO] [UHD] linux; GNU C++ version 11.2.0; Boost_107400; UHD_4.1.0.5-3  
Found device 0  
  default_input = True  
  default_output = True  
  device_id = 0  
  driver = audio  
Aide l = Audio interne Stéréo analogique  
Found device 1  
  default_input = False  
  default_output = False  
  device_id = 1  
  driver = audio  
  label = Monitor of Audio interne Stéréo analogique  
administrateur@rt-mv:~$
```

Donc, le Pluto est bien reconnu par la VM, mais il manque encore le module nécessaire pour l'utiliser directement avec srsRAN. Pour continuer, il faudra installer le support SoapyPlutoSDR ou recompiler srsRAN avec le bon plugin radio.

```
administrateur@rt-mv:~/Téléchargements/srsRAN_4G-master/build$ srsenb --rf.device_name=soapy  
--rf.device_args="driver=plutosdr,hostname=192.168.2.1"  
Active RF plugins: librsran_rf_soapy.so librsran_rf_zmq.so  
Inactive RF plugins:  
--- Software Radio Systems LTE eNodeB ---  
Aide  
Couln't open , trying /home/administrateur/.config/srsran/enb.conf  
Reading configuration file /home/administrateur/.config/srsran/enb.conf...  
Couln't open sib.conf, trying /home/administrateur/.config/srsran/sib.conf  
Couln't open rr.conf, trying /home/administrateur/.config/srsran/rr.conf  
Couln't open rb.conf, trying /home/administrateur/.config/srsran/rb.conf  
WARNING: Could not verify cpu0 scaling governor  
WARNING: Could not verify cpu1 scaling governor  
  
Built in Release mode using 25.10.0.  
  
terminate called after throwing an instance of 'std::bad_alloc'  
what():  std::bad_alloc  
--- command='srsenb --rf.device_name=soapy --rf.device_args=driver=plutosdr,hostname=192.168  
.2.1' version=25.10.0 signal=6 date='12/05/2026 10:49:39' ---  
srsenb(+0x3d308a) [0x5615bcd7608a]  
/lib/x86_64-linux-gnu/libc.so.6(+0x42520) [0x7f5c67842520]  
/lib/x86_64-linux-gnu/libc.so.6(pthread_kill+0x12c) [0x7f5c67896a7c]  
/lib/x86_64-linux-gnu/libc.so.6(raise+0x16) [0x7f5c67842476]  
/lib/x86_64-linux-gnu/libc.so.6(abort+0xd3) [0x7f5c678287f3]  
/lib/x86_64-linux-gnu/libstdc++.so.6(+0xa2bbe) [0x7f5c67ca2bbe]  
/lib/x86_64-linux-gnu/libstdc++.so.6(+0xae24c) [0x7f5c67cae24c]  
/lib/x86_64-linux-gnu/libstdc++.so.6(+0xae2b7) [0x7f5c67cae2b7]
```

Opening SDR

Received S1 Setup Request

Sending S1 Setup Response

```
administrateur@rt-mv: ~/Téléchargements/srsRAN_4G-master/build
administrateur@rt-mv: ~/Téléchargements/srsR... x administrateur@rt-mv: ~/Téléchargements/srsR... x
NC--- command='srseNB --rf.device_name=soapy --rf.device_args=driver=plutosdr,hostname=192.168.2.1' version=25.10.0 signal=11 date='12/05/2026 10:52:04' ---
srseNB(+0x3d308a) [0x55b39238e08a]
/lib/x86_64-linux-gnu/libc.so.6(+0x42520) [0x7fd198c42520]
/lib/x86_64-linux-gnu/libio.so.0(io_channel_disable+0xe) [0x7fd18ce2911e]
/usr/local/lib/SoapySDR/modules0.8/libPlutoSDRSupport.so(+0x12181) [0x7fd18ce51181]
/usr/local/lib/SoapySDR/modules0.8/libPlutoSDRSupport.so(+0x4e4a) [0x7fd18ce4be4a]
/usr/local/lib/SoapySDR/modules0.8/libPlutoSDRSupport.so(+0x4cf3d) [0x7fd18ce4bf3d]
/lib/x86_64-linux-gnu/libSoapySDR.so.0.8(_ZN8SoapySDR6Device6unmakeEPS0_+0x1f4) [0x7fd19969d4f4]
/lib/x86_64-linux-gnu/libSoapySDR.so.0.8(SoapySDRDevice_unmake+0x2d) [0x7fd1996bb2ed]

/usr/local/lib/librsran_rf_soapy.so(rf_soapy_close+0x21) [0x7fd1993511a1]
srseNB(+0x5c8c17) [0x55b392583c17]
srseNB(+0xccc0b0) [0x55b392087cb0]
srseNB(+0xab479) [0x55b392066479]
/lib/x86_64-linux-gnu/libc.so.6(+0x29d90) [0x7fd198c29d90]
/lib/x86_64-linux-gnu/libc.so.6(__libc_start_main+0x80) [0x7fd198c29e40]
srseNB(+0xae005) [0x55b392069005]
srsRAN crashed. Please send this backtrace to the developers ...
--- exiting ---
administrateur@rt-mv:~/Téléchargements/srsRAN_4G-master/build$ sudo rm -f /tmp/enb.log /tmp/epc.log
administrateur@rt-mv:~/Téléchargements/srsRAN_4G-master/build$ sudo srsepc

Built in Release mode using 25.10.0.

--- Software Radio Systems EPC ---

Couldn't open , trying /root/.config/srsran/epc.conf
Reading configuration file /root/.config/srsran/epc.conf...
Couldn't open user_db.csv, trying /root/.config/srsran/user_db.csv
HSS Initialized.
MME S11 Initialized
MME GTP-C Initialized
MME Initialized. MCC: 0xf001, MNC: 0xff01
SPGW GTP-U Initialized.
SPGW S11 Initialized.
SP-GW Initialized.
Received S1 Setup Request.
S1 Setup Request - eNB Name: srseNB01, eNB id: 0x19b
S1 Setup Request - MCC:001, MNC:01
S1 Setup Request - TAC 7, B-PLMN 0xf110
S1 Setup Request - Paging DRX v128
Sending S1 Setup Response
```

proves Pluto eNB started:

Opening PlutoSDR #0 192.168.2.1

==== eNodeB started ====


```
administrateur@rt-mv: ~/Téléchargements/srsRAN_4G-master/build
[INFO] Opening PlutoSDR #0 192.168.2.1...
Setting up Rx stream with 1 channel(s)
[INFO] Using format CF32.
[INFO] Auto setting Buffer Size: 262144
[INFO] Set MTU Size: 262144
Setting up Tx stream with 1 channel(s)
[INFO] Using format CF32.
[INFO] Auto setting TX Buffer Size: 262144
[INFO] Set TX MTU Size: 262144
[INFO] Has direct TX copy: 1
[INFO] Auto setting Buffer Size: 32768
[INFO] Set MTU Size: 32768
[INFO] Auto setting TX Buffer Size: 32768
[INFO] Set TX MTU Size: 32768
Available device sensors:
- xadc_temp0
- xadc_voltage0
- xadc_voltage1
- xadc_voltage2
- xadc_voltage3
- xadc_voltage4
- xadc_voltage5
- xadc_voltage6
- xadc_voltage7
- xadc_voltage8
- adm1177_current0
- adm1177_voltage0
- ad9361-phy_temp0
- ad9361-phy_voltage2
Available sensors for Rx channel 0:
State of gain elements for Rx channel 0 (AGC supported):
- PGA: 40.00 dB
State of gain elements for Tx channel 0 (AGC not supported):
- PGA: 80.00 dB
Rx antenna set to A_BALANCED
Tx antenna set to A

==== eNodeB started ====
Type <t> to view trace
[INFO] Auto setting Buffer Size: 262144
[INFO] Set MTU Size: 262144
[INFO] Auto setting TX Buffer Size: 262144
[INFO] Set TX MTU Size: 262144
Setting frequency: DL=2630.0 Mhz, UL=2510.0 MHz for cc_idx=0 nof_prb=50
Setting frequency: DL=2650.0 Mhz, UL=2530.0 MHz for cc_idx=1 nof_prb=50
[INFO] Has direct RX copy: 1
```

ADLEM pluto pour 2éme vm sae.ROM.01

```
inet6 fe80::e3:95cf:1089:350f/64 scope link noprefixroute
    valid_lft forever preferred_lft forever
4: enx00e0221a65b4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel st
    state UNKNOWN group default qlen 1000
    link/ether 00:e0:22:1a:65:b4 brd ff:ff:ff:ff:ff:ff
    inet 192.168.2.10/24 brd 192.168.2.255 scope global dynamic noprefixroute en
    x00e0221a65b4
        valid_lft 863918sec preferred_lft 863918sec
    PlutoSDR fe80::eaf3:466c:d9b0:4e88/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

```

administrateur@rt-mv:~$ ping 192.168.2.1
PING 192.168.2.1 (192.168.2.1) 56(84) bytes of data.
64 bytes from 192.168.2.1: icmp_seq=1 ttl=64 time=0.702 ms
64 bytes from 192.168.2.1: icmp_seq=2 ttl=64 time=0.527 ms
64 bytes from 192.168.2.1: icmp_seq=3 ttl=64 time=0.317 ms
^C
--- 192.168.2.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2077ms
PlutoSDR g/max/mdev = 0.317/0.515/0.702/0.157 ms
administrateur@rt-mv:~$

```

```

administrateur@rt-mv:~$ lsusb
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 001 Device 003: ID 0456:b673 Analog Devices, Inc. PlutoSDR (ADALM-PLUTO)
Bus 001 Device 002: ID 80ee:0021 VirtualBox USB Tablet
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub

```

34. Bilan final du test avec ADALM-Pluto

Après avoir validé le fonctionnement avec ZeroMQ, nous avons essayé de remplacer la liaison radio virtuelle par une vraie liaison radio à l'aide de deux modules **ADALM-Pluto**. L'objectif était d'utiliser un Pluto côté **VM2** pour l'eNodeB, et un autre Pluto côté **VM1** pour la réception côté UE.

Dans un premier temps, le Pluto a bien été reconnu par la machine virtuelle. L'interface réseau du Pluto était visible avec l'adresse [192.168.2.10/24](#), et le ping vers [192.168.2.1](#) fonctionnait. La commande [iio_info](#) confirmait également que le matériel était détecté. Dans la première version du test, le problème venait du fait que SoapySDR ne détectait pas encore correctement le Pluto, et que srsRAN ne chargeait que le plugin ZMQ.

Après installation et configuration du support SoapySDR/PlutoSDR, srsRAN a ensuite réussi à charger le plugin Soapy. L'eNodeB côté VM2 a pu démarrer avec le Pluto comme interface radio. Les messages suivants ont permis de valider le démarrage côté émission :

```

Opening PlutoSDR #0 192.168.2.1
==== eNodeB started ====
Received S1 Setup Request
Sending S1 Setup Response

```

Cela montre que **VM2 arrivait bien à lancer l'eNodeB avec l'ADALM-Pluto** et que la communication entre [srsENB](#) et [srsEPC](#) était correcte.

Cependant, côté **VM1**, la cellule LTE n'a pas pu être détectée avec [cell_search](#). Même lorsque le Pluto de VM1 était bien reconnu par SoapySDR et libiio, le résultat restait :

```
Found 0 cells
```

Nous avons testé plusieurs solutions pour corriger ce problème :

- vérification du ping vers [192.168.2.1](#) ;

- vérification avec `iio_info` ;
- vérification avec `SoapySDRUtil --probe` ;
- changement de fréquence LTE ;
- passage de la bande 7 à la bande 3 ;
- réduction de la bande passante avec `n_prb = 6` ;
- correction de la configuration PRACH avec `prach_freq_offset = 0` ;
- test de plusieurs valeurs de gain RX ;
- test avec `hostname=192.168.2.1` et avec l'URI USB directe ;
- vérification des ports d'antenne TX/RX.

Malgré ces essais, VM1 ne détectait toujours pas la cellule émise par VM2. Le problème ne semblait donc pas venir de l'EPC ni de l'eNodeB, car VM2 arrivait bien à démarrer l'eNodeB et l'EPC recevait bien la requête S1. Le blocage se situait plutôt au niveau de la **réception radio côté VM1**, probablement lié au chemin RF, aux ports d'antenne, au niveau de signal, au driver `SoapySDR/libiio` ou aux limites de l'utilisation des Pluto dans des machines virtuelles.

35. Conclusion générale

Ce projet a permis de déployer une infrastructure LTE privée avec **srsRAN 4G** sur deux machines virtuelles. La première partie avec **ZeroMQ** a été validée avec succès : l'UE a détecté la cellule LTE, s'est attaché au réseau et a reçu l'adresse IP [172.16.0.2](#) sur l'interface [tun_srsue](#). La communication avec l'EPC a également été validée avec un ping vers [172.16.0.1](#).

La deuxième partie a permis de mettre en place la **Carrier Aggregation** avec deux porteuses en environnement ZMQ. Côté eNodeB, les deux porteuses étaient visibles avec [cc_idx=0](#) et [cc_idx=1](#), et côté UE, la cellule secondaire a été activée avec le message [SCell Activation / Deactivation CMD: 2](#).

Enfin, le passage vers une vraie interface radio avec **ADALM-Pluto** a été commencé. Cette partie a permis de comprendre les contraintes supplémentaires liées au matériel SDR : détection USB, interface IIO, plugin `SoapySDR`, choix des fréquences, réglage des gains et ports d'antenne. Même si la cellule LTE n'a pas été détectée côté VM1, l'eNodeB côté VM2 a bien pu démarrer avec le Pluto, ce qui montre une progression importante vers une transmission radio réelle.

En conclusion, le réseau LTE simulé avec ZMQ est fonctionnel, la Carrier Aggregation a été validée, et la partie ADALM-Pluto a permis d'identifier un blocage réaliste au niveau de la réception RF. Ce projet m'a permis de mieux comprendre l'architecture LTE, le rôle de chaque entité srsRAN et les difficultés rencontrées lors du passage d'une simulation radio à une transmission radio réelle.